MySQL CRUD Tutorials in JavaScript/Node.js: A Step-by-Step Guide

In this tutorial, we will explore how to perform CRUD (Create, Read, Update, Delete) operations with MySQL using JavaScript and Node.js.

Posted on 2023-10-14

MySQL is a popular open-source relational database management system that integrates seamlessly with JavaScript and Node.js applications. In this tutorial, we will explore how to perform CRUD (Create, Read, Update, Delete) operations with MySQL using JavaScript and Node.js. By the end of this article, you'll have a solid understanding of how to interact with a MySQL database from your JavaScript applications.

Prerequisites

1. Node.js installed on your machine.
2. A running MySQL server.
3. Basic knowledge of JavaScript and Node.js.

Step 1: Install Required Packages

Free ChatGPT AI Assistant

Sider Ope

Before we start working with MySQL, let's make sure we have the necessary packages installed. We'll use the `mysql2` package for connecting to the database and executing queries.

```
npm install mysql2
```

Step 2: Set Up the Database Connection

In your Node.js application, you need to establish a connection to the MySQL database. Create a JavaScript file (e.g., `db.js`) and add the following code:

```
const mysql = require("mysql2")

const connection = mysql.createConnection({
  host: "your-database-host",
  user: "your-username",
  password: "your-password",
```

```
  database: "your-database-name"
})

connection.connect((err) => {
  if (err) {
    console.error("Error connecting to MySQL:", err)
    return
  }
  console.log("Connected to MySQL")
})

module.exports = connection
```

Replace `'your-database-host'`, `'your-username'`, `'your-password'`, and `'your-database-name'` with your MySQL server details.

Free ChatGPT AI Assistant

Sider Ope

Step 3: Create a New Record (Create - C)

Now, let's create a function to insert data into a MySQL table. Create a JavaScript file (e.g., `create.js`) and add the following code:

```
const connection = require("./db")

function createRecord(name, email) {
  const sql = "INSERT INTO users (name, email) VALUES (?, ?)"
  connection.query(sql, [name, email], (err, result) => {
    if (err) {
      console.error("Error creating record:", err)
      return
    }
    console.log(`New record added with ID: ${result.insertId}`)
  })
}

createRecord("John Doe", "john@example.com")
```

This code defines a function `createRecord` that inserts a new user into a 'users' table.

Free ChatGPT Browser Extensi

Sider Ope

Step 4: Read Data (Read - R)

To read data from the database, create another JavaScript file (e.g., `read.js`) with the following code:

```
const connection = require("./db")

function readRecords() {
  const sql = "SELECT * FROM users"
  connection.query(sql, (err, rows) => {
    if (err) {
      console.error("Error reading records:", err)
      return
    }
    rows.forEach((row) => {
      console.log(`ID: ${row.id}, Name: ${row.name}, Email: ${row.email}`)
    })
  })
}

readRecords()
```

This code defines a function `readRecords` that retrieves all user records from the 'users' table and prints them to the console.

Free ChatGPT AI Assistant

Sider Ope

Step 5: Update Data (Update - U)

To update existing records, create a JavaScript file (e.g., `update.js`) with the following code:

```
const connection = require("./db")

function updateRecord(id, newName) {
```

```
  const sql = "UPDATE users SET name = ? WHERE id = ?"
  connection.query(sql, [newName, id], (err, result) => {
    if (err) {
      console.error("Error updating record:", err)
      return
    }
    console.log(`Record updated: ${result.affectedRows} rows affected`)
  })
}

updateRecord(1, "Jane Doe")
```

This code defines a function `updateRecord` that updates a user's name based on their ID.

Free ChatGPT Browser Extensi

Sider Ope

Step 6: Delete Data (Delete - D)

To delete records, create a JavaScript file (e.g., `delete.js`) with the following code:

```
const connection = require("./db")

function deleteRecord(id) {
  const sql = "DELETE FROM users WHERE id = ?"
  connection.query(sql, [id], (err, result) => {
    if (err) {
      console.error("Error deleting record:", err)
      return
    }
    console.log(`Record deleted: ${result.affectedRows} rows affected`)
  })
}

deleteRecord(1)
```

This code defines a function `deleteRecord` that deletes a user record based on their ID.

SQLiz

In this article, we've covered the fundamental CRUD operations using MySQL with JavaScript and Node.js. You've learned how to establish a database connection, create, read, update, and delete records. This knowledge will empower you to build more complex applications that interact with MySQL databases seamlessly. Remember to handle errors gracefully and adapt these examples to suit your specific project requirements.

← Oracle CRUD Tutorials in JavaScript/Node.js: A Step-by-Step Guide SQLite CRUD Tutorials in JavaScript/Node.js: A Step-by-Step Guide →